

Patuakhali Science and Technology University

Faculty of Computer Science and Engineering

Lab Report

Course Code: CCE 314

Course Title: Computer Networks Sessional

(Numerical Methods Algorithms)

Submitted From:

Sandipta Saha

ID: 2102056

Reg: 10183

Semester: 5 (Level-3, Sem-1)

Submitted To:

Prof. Dr. Md Samsuzzaman

Professor

Dept. of CCE

Patuakhali Science and Technology

Univ.

Golam Md. Muradul Bashir

Professor

Dept. of CCE

Patuakhali Science and Technology

Univ.

Submission Date: Sat, 13 November 2025

Contents

1	Algebraic & Linear System Methods	2
1.1	Lab Problem 1: Cramer's Rule	2
1.2	Lab Problem 2: Gauss Elimination Method	2
1.3	Lab Problem 3: Gauss-Jordan Elimination	3
1.4	Lab Problem 4: Jacobi Iteration Method	4
2	Root-Finding Methods	4
2.1	Lab Problem 5: Bisection Method	4
2.2	Lab Problem 6: False Position Method	5
2.3	Lab Problem 7: Newton-Raphson Method	5
3	Interpolation & Approximation	5
3.1	Lab Problem 8: Newton's Divided Difference	5
3.2	Lab Problem 9: Lagrange Interpolation	6
4	Numerical Integration	6
4.1	Lab Problem 10: Trapezoidal Rule	6
4.2	Lab Problem 11: Simpson's 1/3 Rule	6
4.3	Lab Problem 12: Simpson's 3/8 Rule	6
5	Differential Equation Solvers	7
5.1	Lab Problem 13: Euler's Method	7
5.2	Lab Problem 14: Runge-Kutta (RK4) Method	7
5.3	Lab Problem 15: Milne's Predictor-Corrector Method	7
5.4	Lab Problem 16: Picard's Method	8
6	Regression & Curve Fitting	8
6.1	Lab Problem 17: Least Squares Method (General)	8
6.2	Lab Problem 18: Linear Regression	8
6.3	Lab Problem 19: Polynomial Regression	8
6.4	Lab Problem 20: Logistic Regression	9
7	Optimization & Learning	9
7.1	Lab Problem 21: Stochastic Gradient Descent (SGD)	9
7.2	Lab Problem 22: Complete System Solver (SciPy)	9

1 Algebraic & Linear System Methods

1.1 Lab Problem 1: Cramer's Rule

Objective: To solve a system of linear equations $Ax = b$ using Cramer's Rule, which relies on the computation of determinants.

Method: Given a system of n linear equations $Ax = b$, the solution for each unknown x_i is:

$$x_i = \frac{|A_i|}{|A|}$$

Where $|A|$ is the determinant of A , and $|A_i|$ is the determinant of the matrix formed by replacing the i -th column of A with b .

Python Implementation:

```

1 import numpy as np
2
3 def cramers_rule(A, b):
4     n = A.shape[0]
5     if A.shape[1] != n:
6         raise ValueError("Matrix A must be square.")
7
8     det_A = np.linalg.det(A)
9     if det_A == 0:
10         print("System has no unique solution.")
11         return None
12
13     solutions = np.zeros(n)
14     for i in range(n):
15         A_i = A.copy()
16         A_i[:, i] = b
17         det_A_i = np.linalg.det(A_i)
18         solutions[i] = det_A_i / det_A
19
20     return solutions

```

$$2x_1 + x_2 - x_3 = 8$$

[cite_{start}]Example Problem:[cite : 77–92]–3x₁ – x₂ + 2x₃ = –11 Coefficient Ma-

$$-2x_1 + x_2 + 2x_3 = -3$$

trix A:

$$\begin{bmatrix} 2 & 1 & -1 \\ -3 & -1 & 2 \\ -2 & 1 & 2 \end{bmatrix}$$

Constant Vector b: [8, –11, –3]

Solution x (x₁, x₂, x₃): [2., 3., –1.]

1.2 Lab Problem 2: Gauss Elimination Method

Objective: To solve $Ax = b$ by transforming the augmented matrix into row-echelon form and using back substitution.

Algorithm:

1. Forward Elimination: Introduce zeros below the main diagonal.

2. Back Substitution: Solve for unknowns from x_n to x_1 .

Python Implementation:

```

1 def gauss_elimination(A, b):
2     n = len(b)
3     M = np.hstack([A.astype(float), b.reshape(-1, 1).astype(float)])
4
5     # Forward Elimination
6     for i in range(n):
7         pivot = M[i, i]
8         if pivot == 0:
9             for k in range(i + 1, n):
10                 if M[k, i] != 0:
11                     M[[i, k]] = M[[k, i]] # Swap rows
12                     pivot = M[i, i]
13                     break
14             else:
15                 raise ValueError("Matrix is singular.")
16
17         for j in range(i + 1, n):
18             factor = M[j, i] / pivot
19             M[j, i:] = M[j, i:] - factor * M[i, i:]
20
21     # Back Substitution
22     x = np.zeros(n)
23     for i in range(n - 1, -1, -1):
24         dot_sum = np.dot(M[i, i+1:n], x[i+1:n])
25         x[i] = (M[i, n] - dot_sum) / M[i, i]
26
27     return x

```

[cite_{start}]Output:[cite : 162]
 Solution $x(x_1, x_2, x_3)$: [2., 3., 1.]

1.3 Lab Problem 3: Gauss-Jordan Elimination

Objective: To solve $Ax = b$ by transforming the augmented matrix into reduced row-echelon form.

Python Implementation:

```

1 def gauss_jordan(A, b):
2     n = len(b)
3     M = np.hstack([A.astype(float), b.reshape(-1, 1).astype(float)])
4
5     for i in range(n):
6         max_row = i + np.argmax(np.abs(M[i:, i])) # Partial pivoting
7         M[[i, max_row]] = M[[max_row, i]]
8
9         M[i, :] = M[i, :] / M[i, i] # Normalize pivot
10
11        for j in range(n):
12            if i != j:
13                M[j, :] = M[j, :] - M[j, i] * M[i, :] # Eliminate
14
15    return M[:, n]

```

1.4 Lab Problem 4: Jacobi Iteration Method

Objective: Solve $Ax = b$ using iterative approach for diagonally dominant systems.

Python Implementation:

```

1 def jacobi_iteration(A, b, max_iter=100, tol=1e-6):
2     n = len(b)
3     x = np.zeros(n)
4     x_new = np.zeros(n)
5
6     for k in range(max_iter):
7         for i in range(n):
8             sigma = 0
9             for j in range(n):
10                if i != j:
11                    sigma += A[i, j] * x[j]
12                x_new[i] = (b[i] - sigma) / A[i, i]
13
14            if np.linalg.norm(x_new - x, ord=np.inf) < tol:
15                return x_new, k + 1
16            x = x_new.copy()
17
18    return None, max_iter

```

[cite_{start}]Output:[cite : 238 – 239]

Solution x : [1., 2., -1.]

Converged in 10 iterations.

2 Root-Finding Methods

2.1 Lab Problem 5: Bisection Method

Objective: Find root of $f(x)$ in $[a, b]$.

Python Implementation:

```

1 def bisection_method(f, a, b, tol=1e-6, max_iter=100):
2     if f(a) * f(b) >= 0:
3         print("f(a) and f(b) must have opposite signs.")
4         return None, 0
5
6     for i in range(max_iter):
7         c = (a + b) / 2.0
8         if (b - a) / 2.0 < tol or f(c) == 0:
9             return c, i + 1
10        if f(a) * f(c) < 0:
11            b = c
12        else:
13            a = c
14
15    return (a + b) / 2.0, max_iter

```

[cite_{start}]Output:[cite : 271 – 274]

Function : $f(x) = x^3 - x - 2$

Interval: [1.0, 2.0]

Root found: 1.521380

Found in 20 iterations.

2.2 Lab Problem 6: False Position Method

Method: Uses linear interpolation (secant line).

$$c = b - \frac{f(b)(a - b)}{f(a) - f(b)}$$

Python Implementation:

```

1 def false_position_method(f, a, b, tol=1e-6, max_iter=100):
2     for i in range(max_iter):
3         c = a - (f(a) * (b - a)) / (f(b) - f(a))
4         if abs(f(c)) < tol:
5             return c, i + 1
6         if f(a) * f(c) < 0:
7             b = c
8         else:
9             a = c
10    return c, max_iter

```

2.3 Lab Problem 7: Newton-Raphson Method

Objective: Find root using tangent lines. $x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)}$.

Python Implementation:

```

1 def newton_raphson_method(f, f_prime, x0, tol=1e-6, max_iter=100):
2     x_n = x0
3     for i in range(max_iter):
4         f_n = f(x_n)
5         f_prime_n = f_prime(x_n)
6         if f_prime_n == 0:
7             print("Derivative is zero.")
8             return None, i
9         x_n_plus_1 = x_n - f_n / f_prime_n
10        if abs(x_n_plus_1 - x_n) < tol:
11            return x_n_plus_1, i + 1
12        x_n = x_n_plus_1
13    return x_n, max_iter

```

[cite_{start}]Output:[cite : 330 – 332]

Initial guess : 1.5

Root found : 1.521380

Found in 3 iterations.

3 Interpolation & Approximation

3.1 Lab Problem 8: Newton's Divided Difference

Python Implementation:

```

1 def get_newton_coeffs(x, y):
2     n = len(x)
3     coef = np.zeros([n, n])
4     coef[:, 0] = y
5     for j in range(1, n):
6         for i in range(n - j):

```

```

7         coef[i][j] = (coef[i+1][j-1] - coef[i][j-1]) / (x[i+j] - x[
            i])
8     return coef[0, :]

```

3.2 Lab Problem 9: Lagrange Interpolation

Python Implementation:

```

1 def lagrange_interpolation(x_nodes, y_nodes, val):
2     n = len(x_nodes)
3     P = 0.0
4     for j in range(n):
5         L_j = 1.0
6         for i in range(n):
7             if i != j:
8                 L_j = L_j * (val - x_nodes[i]) / (x_nodes[j] - x_nodes[
                i])
9         P = P + y_nodes[j] * L_j
10    return P

```

[cite_{start}]Output:[cite : 377]

DataPoints : (0, 0), (1, 1.7183), (2, 5.3891), (3, 19.0855)

Estimated value at x=1.5: 3.324275

4 Numerical Integration

4.1 Lab Problem 10: Trapezoidal Rule

Python Implementation:

```

1 def trapezoidal_rule(f, a, b, n):
2     h = (b - a) / n
3     x = np.linspace(a, b, n + 1)
4     y = f(x)
5     I = (h/2.0) * (y[0] + 2*np.sum(y[1:n]) + y[n])
6     return I

```

4.2 Lab Problem 11: Simpson's 1/3 Rule

Python Implementation:

```

1 def simpsons_1_3_rule(f, a, b, n):
2     if n % 2 != 0:
3         raise ValueError("n must be an even integer.")
4     h = (b - a) / n
5     x = np.linspace(a, b, n + 1)
6     y = f(x)
7     I = (h/3.0) * (y[0] + 4*np.sum(y[1:n:2]) + 2*np.sum(y[2:n:2]) + y[n])
8     return I

```

4.3 Lab Problem 12: Simpson's 3/8 Rule

Python Implementation:

```
1 def simpsons_3_8_rule(f, a, b, n):
2     if n % 3 != 0:
3         raise ValueError("n must be a multiple of 3.")
4     h = (b - a) / n
5     x = np.linspace(a, b, n + 1)
6     y = f(x)
7     I = (3*h/8.0)*(y[0] + 3*np.sum(y[1:n:3]) + 3*np.sum(y[2:n:3]) + 2*
8         np.sum(y[3:n:3]) + y[n])
9     return I
```

5 Differential Equation Solvers

5.1 Lab Problem 13: Euler's Method

Python Implementation:

```
1 def eulers_method(f, x0, y0, h, x_target):
2     x = x0; y = y0
3     n = int((x_target - x0) / h)
4     for i in range(n):
5         y = y + h * f(x, y)
6         x = x + h
7     return y
```

5.2 Lab Problem 14: Runge-Kutta (RK4) Method

Python Implementation:

```
1 def rk4_method(f, x0, y0, h, x_target):
2     x, y = x0, y0
3     n = int((x_target - x0) / h)
4     for i in range(n):
5         k1 = f(x, y)
6         k2 = f(x + 0.5*h, y + 0.5*h*k1)
7         k3 = f(x + 0.5*h, y + 0.5*h*k2)
8         k4 = f(x + h, y + h*k3)
9         y = y + (h/6.0)*(k1 + 2*k2 + 2*k3 + k4)
10        x = x + h
11    return y
```

5.3 Lab Problem 15: Milne's Predictor-Corrector Method

Python Implementation:

```
1 def milne_method(f, x_nodes, y_nodes):
2     h = x_nodes[1] - x_nodes[0]
3     f0 = f(x_nodes[0], y_nodes[0])
4     f1 = f(x_nodes[1], y_nodes[1])
5     f2 = f(x_nodes[2], y_nodes[2])
6     f3 = f(x_nodes[3], y_nodes[3])
7
8     # Predictor
9     y4_pred = y_nodes[0] + (4*h/3.0)*(2*f1 - f2 + 2*f3)
10    x4 = x_nodes[3] + h
11
```



```

12 # Corrector
13 f4_pred = f(x4, y4_pred)
14 y4_corr = y_nodes[2] + (h/3.0)*(f2 + 4*f3 + f4_pred)
15 return x4, y4_corr

```

5.4 Lab Problem 16: Picard's Method

Python Implementation:

```

1 import sympy
2 from sympy.abc import x, t
3
4 def picards_method(f_xy, x0, y0, iterations=3):
5     y = [None] * (iterations + 1)
6     y[0] = sympy.sympify(y0)
7     for i in range(iterations):
8         f_t_y_i = f_xy.subs(x, t).subs(sympy.Symbol('y'), y[i].subs(x,
9         t))
10         integral = sympy.integrate(f_t_y_i, (t, x0, x))
11         y[i+1] = sympy.sympify(y0) + integral
12     return y[iterations]

```

6 Regression & Curve Fitting

6.1 Lab Problem 17: Least Squares Method (General)

Objective: Minimize the sum of squared errors $S = \sum (y_i - g(x_i))^2$ to fit a model.

6.2 Lab Problem 18: Linear Regression

Python Implementation:

```

1 def linear_regression(x, y):
2     n = len(x)
3     sum_x = np.sum(x); sum_y = np.sum(y)
4     sum_x_sq = np.sum(x**2); sum_xy = np.sum(x*y)
5     A = np.array([[n, sum_x], [sum_x, sum_x_sq]])
6     b = np.array([sum_y, sum_xy])
7     try:
8         a0, a1 = np.linalg.solve(A, b)
9         return a0, a1
10    except np.linalg.LinAlgError:
11        return None, None

```

6.3 Lab Problem 19: Polynomial Regression

Python Implementation:

```

1 def polynomial_regression(x, y, degree):
2     coefficients = np.polyfit(x, y, degree)
3     polynomial = np.poly1d(coefficients)
4     return coefficients, polynomial

```

6.4 Lab Problem 20: Logistic Regression

Python Implementation:

```

1 from sklearn.linear_model import LogisticRegression
2 def logistic_regression(X, y):
3     model = LogisticRegression(solver='liblinear')
4     model.fit(X, y)
5     return model

```

7 Optimization & Learning

7.1 Lab Problem 21: Stochastic Gradient Descent (SGD)

Objective: Optimize parameters by minimizing error one data point at a time. $a_j = a_j - \eta \times \text{gradient}$.

Python Implementation:

```

1 from sklearn.preprocessing import StandardScaler
2
3 def sgd_linear_regression(x, y, eta=0.01, epochs=100):
4     scaler = StandardScaler()
5     x_scaled = scaler.fit_transform(x.reshape(-1, 1)).ravel()
6     a0 = 0.0; a1 = 0.0
7     n = len(x)
8
9     for epoch in range(epochs):
10         indices = np.random.permutation(n)
11         for i in indices:
12             xi = x_scaled[i]
13             yi = y[i]
14             y_pred = a0 + a1 * xi
15             error = y_pred - yi
16             grad_a0 = error
17             grad_a1 = error * xi
18             a0 = a0 - eta * grad_a0
19             a1 = a1 - eta * grad_a1
20     return a0, a1

```

7.2 Lab Problem 22: Complete System Solver (SciPy)

Objective: Solving standard problems using the 'scipy' library.

[cite_{start}]**Python Implementation:**[cite : 636 – 662]

1. Linear System $A_{lin} = \text{np.array}([[2, 1, -1], [-3, -1, 2], [-2, 1, 2]])$ $b_{lin} = \text{np.array}([8, -11, -3])$ $sol_{lin} = \text{linalg.solve}(A_{lin}, b_{lin})$ $\text{print}(f"LinearSolution : sol_{lin}")$
2. Root-Finding $\text{def } f_{root}(x) : \text{return } x**3 - x - 2$ $sol_{root} = \text{optimize.root_scalar}(f_{root}, \text{bracket} = [1, 2], \text{method} = 'brentq')$ $\text{print}(f"Root : sol_{root}.root")$
3. Integration $sol_{int}, err = \text{integrate.quad}(\text{np.sin}, 0, \text{np.pi})$ $\text{print}(f"Integral : sol_{int}")$
4. ODE $\text{def } f_{ode}(x, y) : \text{return } x + y$ $sol_{ode} = \text{integrate.solve_vp}(f_{ode}, [0.0, 1.0], [1.0])$ $\text{print}(f"ODESolution at end : sol_{ode}.y[0][-1]")$